

UNIVERSIDAD DE COSTA RICA
SEDE PACÍFICO

ALGORITMOS Y ESTRUCTURAD DE DATOS

INFORMÁTICA
EMPRESARIAL

MATERIAL DIDÁCTICO



```
public class Nodo <T>{  
    private T dato;  
    private Nodo<T> siguiente;  
  
    public Nodo (T dato){  
        siguiente=null;  
        this.dato=dato;  
    }  
} // fin de la clase
```

ESTE TEXTO ESTÁ DIRIGIDO A QUIENES SE ESFUERZAN
POR APRENDER DÍA A DÍA, SIN IMPORTAR LAS
CONDICIONES ADVERSAS QUE IMPEREN

Tema 3

Tipos de datos abstractos

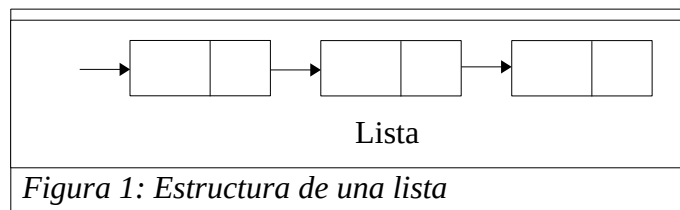
Karol Sánchez y Marcos Venegas

TDA Listas

Las listas constituyen una estructura lineal flexible que permite insertar, buscar y eliminar elementos en cualquier posición de la lista. Las listas también pueden unirse entre sí o dividirse en sublistas; se presentan de manera rutinaria en aplicaciones como:

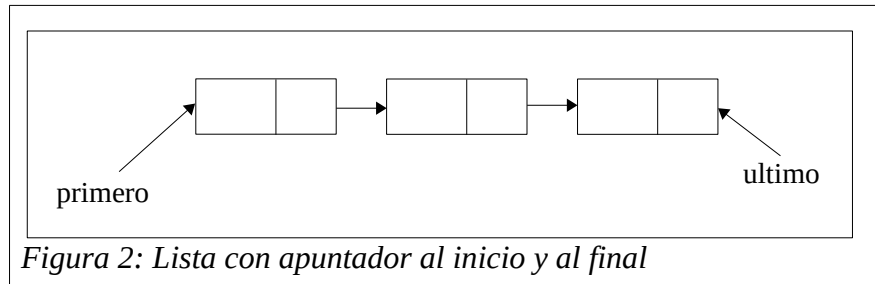
- recuperación de información.
- traducción de lenguajes de programación.
- simulación.

Una lista es una estructura de datos dinámica, donde la cantidad de nodos puede variar rápidamente en su ejecución, aumentando por las inserciones y disminuyendo por la supresión de nodos.



Las inserciones o supresiones se pueden realizar por cualquier punto de la lista: por el inicio (primero), por el final (último), antes o después de un nodo determinado de la lista, o por su posición. Además,

también se pueden eliminar los nodos dependiendo del campo donde se encuentre la información o el dato que se desea suprimir de la lista.



Las listas se utilizan para almacenar información del mismo tipo, con la característica de que puede contener un número indefinido de elementos y que estos elementos mantienen un orden evidente. Este ordenamiento implica que cada elemento (cada nodo de la lista) contiene la dirección del siguiente elemento por medio de un enlace.

Matemáticamente, una lista se define como una secuencia de cero o más elementos de un tipo definido. A menudo se representa una lista como una sucesión de elementos separados por comas $a_0, a_1, \dots, a_n$; donde $n \geq 0$ y cada a_i es del tipo definido. Al número n de elementos se le llama longitud de la lista.

Una propiedad importante de una lista sencilla es que sus elementos están ordenados en forma lineal de acuerdo con sus posiciones en la misma y su comportamiento es independiente del tipo de datos almacenado.

Para formar un tipo de datos abstractos a partir de la noción matemática de lista, se debe definir un conjunto de operaciones con objetos de tipo **Lista**.

Operaciones del TDA Lista

En estas operaciones, cada lista es una serie de objetos de tipo definido, donde x es un objeto de ese tipo y p es la posición de cada objeto.

Un tipo de datos abstracto de la familia Lista incluye a menudo las ocho operaciones siguientes:

1. ***inserta(x, p)*** = Esta función inserta un elemento (x) en una posición (p) definida de la lista, pasando los elementos de la posición p y siguientes a la posición inmediatamente posterior. Esto quiere decir que si la lista es $a_1, a_2, \dots, a_n$, se convierte en $a_1, a_2, \dots, a_{p-1}, x, a_p, \dots, a_n$. Si p es mayor a la posición final, entonces la lista se convierte en $a_1, a_2, \dots, a_n, x$.
2. ***localiza(x)*** = Esta función devuelve la posición del elemento x en la lista. Si el elemento x figura más de una vez, la posición de la primera aparición del elemento x es la que se devuelve. Si el elemento no se encuentra se devuelve -1.
3. ***recupera(p)*** = Esta función devuelve el elemento que está en la posición p de la lista. Si el elemento no se encuentra se devuelve -1.
4. ***suprime(p)*** = Esta función elimina el elemento en la posición p de la lista. Si la lista es $a_1, a_2, \dots, a_n$, la lista se convierte en $a_1, a_2, \dots, a_{p-1}, a_{p+1}, \dots, a_n$.
5. ***anula()*** = Esta función ocasiona que la lista se convierta en la lista vacía.
6. ***primero()*** = Esta función devuelve el valor del elemento que se encuentra en la primera posición de la lista.
7. ***imprimirLista()*** = Imprime los elementos de la lista en su orden de aparición.
8. ***esVacía()*** = Devuelve un ***true*** si la lista se encuentra vacía y un ***false*** en caso contrario.

Realización de listas mediante arreglos

En la realización de una lista mediante arreglos, los elementos de ésta se almacenan en celdas contiguas de un arreglo. Esta representación permite recorrer con facilidad una lista y agregarle elementos nuevos al final. Pero insertar un elemento en la mitad de la lista obliga a desplazarse una posición dentro del arreglo a todos los elementos que siguen al nuevo elemento para concederle espacio. De la misma forma, la eliminación de un elemento, excepto el último, requiere desplazamientos de elementos para llenar de nuevo el vacío formado.

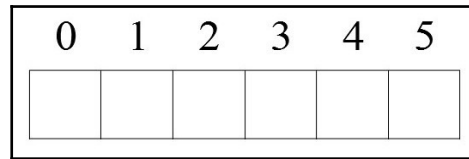


Figura 3: TDA Lista con vectores

Realización de listas mediante punteros

La segunda forma de realización de listas, celdas enlazadas sencillas, utiliza apuntadores para enlazar elementos consecutivos. Esta implantación permite eludir el empleo de memoria contigua para almacenar una lista y, por tanto, también elude los desplazamientos de elementos para hacer inserciones o rellenar vacíos creados por la eliminación de elementos.

En esta representación, una lista está formada por celdas; cada celda contiene un elemento de la lista y un apuntador a la siguiente celda.

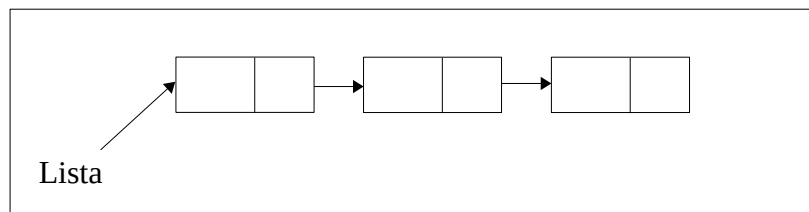


Figura 4: TDA Lista con punteros

Bibliografía

Aho, A. V., Hopcroft, J. E., & Ullman, J. D. (1988). Estructuras de datos y algoritmos (Vol. 1). Addison-Wesley Iberoamericana.

Birtwistle, G. M., Dahl, O. J., Myhrhaug, B., & Nygaard, K. (1973) SIMULA BEGIN. Filadelfia, Pensilvania. Auerbach Press

Cook, W. R. (2009, October). On understanding data abstraction, revisited. In Proceedings of the 24th ACM SIGPLAN conference on Object oriented programming systems languages and applications (pp. 557-572).

Guttag, J. V. (1975). The specification and application to programming of abstract data types. University of Toronto.

Joyanes, L., Martínez, I., & Zahonero, I. (2008). Estructuras de datos en java. McGraw Hill.

Liskov, B., & Zilles, S. (1974). Programming with abstract data types. ACM Sigplan Notices, 9(4), 50-59.